

User Documentation

Zero-Downtime Linux Updates

This document explains how to install, configure, and operate the **Orchestrator** agent on an Edge IPC device.

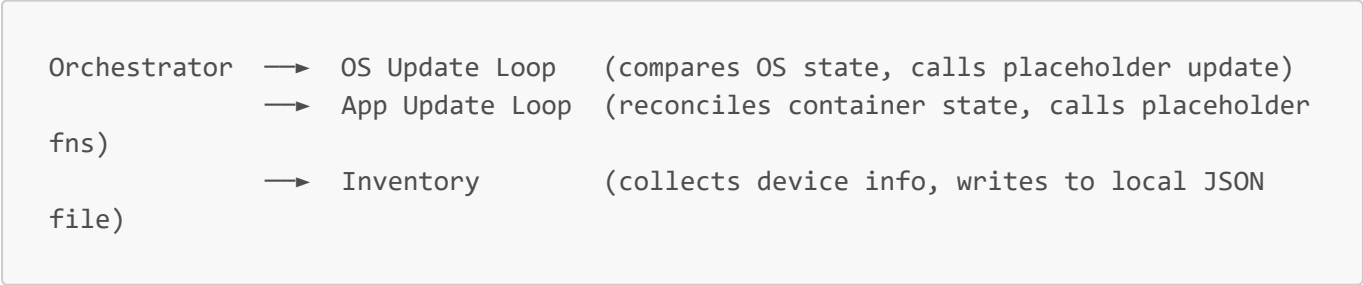
Table of Contents

- 1. [Overview](#)
- 2. [Prerequisites](#)
- 3. [Installation](#)
- 4. [Configuration](#)
- 5. [Running the Orchestrator](#)
- 6. [CLI Reference](#)
- 7. [Self-Check / Health Check](#)
- 8. [Inventory](#)
- 9. [Logging & Verbosity](#)
- 10. [Running as a systemd Service](#)
- 11. [Rollback & Error Recovery](#)
- 12. [API Mock Server](#)

Overview

The **Orchestrator** is a background agent that runs on each Edge IPC. It periodically compares the desired state of the OS and application containers against the host's current state and is designed to trigger updates when they differ.

Current status: The update loops and reconciliation logic are in place, but the actual `bootc/rpm-ostree` OS update commands and Podman container management calls are **placeholder stubs** — they log output but do not yet perform real updates.



Prerequisites

Requirement	Notes
Linux	Any distro; rpm-ostree/bootc compatible OS recommended for future update support

Requirement	Notes
<code>bootc</code>	Optional — used for inventory collection (<code>bootc status</code>); update commands not yet wired
<code>podman</code>	Optional — used for inventory collection (<code>podman ps</code>); container management not yet wired
Network access to Cloud API	HTTPS required (see Configuration)
Rust toolchain	Only needed if building from source

Installation

Pre-built binary

Copy the `amos-orchestrator` binary to the target device and place it in `/usr/local/bin/`:

```
sudo cp amos-orchestrator /usr/local/bin/amos-orchestrator
sudo chmod +x /usr/local/bin/amos-orchestrator
```

Building from source

See the [Build & Deploy Documentation](#) for full build instructions.

Configuration

The Orchestrator reads its configuration from a TOML file. By default it looks for `config.toml` in the working directory. A custom path can be passed via `--config`.

A ready-to-use template with inline documentation for all available options is provided at `orchestrator/config.example.toml`. Copy it and adjust the values for your environment:

```
cp orchestrator/config.example.toml config.toml
```

Environment variable overrides

All config values can be overridden with environment variables prefixed `APP_`:

Environment variable	Config key	Description
<code>APP_CLOUD_URL</code>	<code>cloud_url</code>	Cloud API base URL
<code>APP_POLL_INTERVAL_SECS</code>	<code>poll_interval_secs</code>	Poll frequency in seconds
<code>APP_INVENTORY_PATH</code>	<code>inventory_path</code>	Inventory output file path

Environment variable	Config key	Description
<code>https_proxy</code>	—	HTTPS proxy URL (request default)

Validation rules

- `cloud_url` **must** begin with `http://` or `https://`.
- `poll_interval_secs` **must** be ≥ 1 .

Running the Orchestrator

```
# Use default config.toml in current directory
amos-orchestrator

# Specify a custom config file
amos-orchestrator --config /etc/amos/config.toml

# Enable verbose logging (repeat for more verbosity)
amos-orchestrator -d          # debug
amos-orchestrator -dd        # trace
```

Stop the agent with **Ctrl+C** (SIGINT).

CLI Reference

Usage: amos-orchestrator [OPTIONS]

Options:

<code>-s, --self-check</code>	Run self-check instead of the main loop
<code>-c, --config <FILE></code>	Path to a custom config file
<code>-d, --debug...</code>	Increase log verbosity (repeatable: <code>-d</code> , <code>-dd</code>)
<code>-h, --help</code>	Print help
<code>-V, --version</code>	Print version

Self-Check / Health Check

The `--self-check` flag validates the system configuration and inventory tooling without starting the main loop. Use it to verify the agent is correctly set up:

```
amos-orchestrator --self-check
amos-orchestrator --self-check --config /etc/amos/config.toml
```

Exit codes:

- 0 — all checks passed
- 1 — one or more checks failed (details printed to stderr)

Inventory

On startup, the Orchestrator collects a **device inventory** and writes it as a JSON file to the path defined by `inventory_path`. The inventory includes:

Section	Contents
<code>system</code>	Hostname, OS name/version, kernel version
<code>deployments</code>	rpm-ostree deployment info (checksum, version, booted/staged flags)
<code>bootc_status</code>	Booted, staged, and rollback image info
<code>applications</code>	Running application container names and versions

If a section cannot be collected (e.g. `bootc` is not installed), the field is marked `"status": "unavailable"` with a reason — the rest of the inventory is still written.

Logging & Verbosity

Log output is written to **stderr**. Default level is `WARN`. Use `-d` / `-dd` flags to increase verbosity:

Flags	Level
<code>(none)</code>	<code>WARN</code>
<code>-d</code>	<code>INFO</code>
<code>-dd</code>	<code>DEBUG</code>
<code>-ddd</code>	<code>TRACE</code>

Advanced: The standard `RUST_LOG` environment variable can be used for granular per-module filtering (e.g. `RUST_LOG=amos_orchestrator=debug`). When `-d` flags are provided on the command line they take precedence over `RUST_LOG`.

Running as a systemd Service

A systemd unit file is provided in `rootc-build/orchestrator.service`. To install it:

```
sudo cp rootc-build/orchestrator.service /etc/systemd/system/  
sudo systemctl daemon-reload  
sudo systemctl enable --now amos-orchestrator
```

Check status and logs:

```
sudo systemctl status amos-orchestrator
sudo journalctl -u amos-orchestrator -f
```

Rollback & Error Recovery

Future release: Automated rollback and error recovery are not yet implemented.

If an update causes a problem you can manually trigger a rollback using the standard OS tooling:

```
# OS rollback via bootc
sudo bootc rollback

# OS rollback via rpm-ostree
sudo rpm-ostree rollback
```

For application containers, use `podman` to switch back to the previous image tag manually. Automated rollback support will be added in a future sprint.

API Mock Server

During development or testing, a local mock server (`amos-api-mock-server`) can stand in for a real Cloud API. It serves a static catalog at `GET /v1/catalog` and static download assets from a local `assets/` directory.

Note: The mock server runs on plain HTTP (port 80). The Orchestrator config accepts both `http://` and `https://` URLs, so you can point it directly at `http://localhost` for local testing without a reverse proxy.

```
# Start mock server on port 80 (requires root)
sudo ./amos-api-mock-server
```

Tip: To avoid `sudo`, edit `api-mock-server/src/main.rs` to bind to a high port (e.g. `8080`) and rebuild. Then set `cloud_url = "http://localhost:8080/api/v1"` in your config.

The catalog response from the mock server looks like:

```
[
  { "name": "os", "version": "1.2.3", "url": "ghcr.io/amosproj/...", "signature":
    "AAAA..." },
  { "name": "app", "version": "4.5.6", "url": "/v1/download/app4.5.6",
    "signature": "AAAA..." }
]
```

API Reference

All routes are served under `/v1`. Fields marked `*` are required.

Device Summaries *(read-only)*

Method	Path	Description
GET	<code>/v1/devices/summary</code>	List device summaries. Query: <code>?tenant_id=<id></code>
GET	<code>/v1/devices/{id}/summary</code>	Get a single device summary

Tenants

Method	Path	Description
GET	<code>/v1/tenants</code>	List all tenants
POST	<code>/v1/tenants</code>	Create — body: <code>{ name*, description }</code>
GET	<code>/v1/tenants/{id}</code>	Get by ID
PUT	<code>/v1/tenants/{id}</code>	Replace by ID
DELETE	<code>/v1/tenants/{id}</code>	Delete — 204 on success

Groups

Method	Path	Description
GET	<code>/v1/groups</code>	List all groups
POST	<code>/v1/groups</code>	Create — body: <code>{ name* }</code>
GET	<code>/v1/groups/{id}</code>	Get by ID
PUT	<code>/v1/groups/{id}</code>	Replace by ID
DELETE	<code>/v1/groups/{id}</code>	Delete — 204 on success

Devices

Method	Path	Description
GET	<code>/v1/devices</code>	List devices. Query: <code>?group_id=<id>&tenant_id=<id></code>
POST	<code>/v1/devices</code>	Create — body: <code>{ uuid*, hostname*, tenant_id, group_id }</code>
GET	<code>/v1/devices/{id}</code>	Get by ID
PUT	<code>/v1/devices/{id}</code>	Replace by ID
DELETE	<code>/v1/devices/{id}</code>	Delete — 204 on success

Applications

Method	Path	Description
GET	/v1/applications	List all applications
POST	/v1/applications	Create — body: { name*, description }
GET	/v1/applications/{id}	Get by ID
PUT	/v1/applications/{id}	Replace by ID
DELETE	/v1/applications/{id}	Delete — 204 on success

Application Configs

Method	Path	Description
GET	/v1/app-configs	List configs. Query: ?application_id=<id>
POST	/v1/app-configs	Create — body: { application_id, image*, config, comment }
GET	/v1/app-configs/{id}	Get by ID
PUT	/v1/app-configs/{id}	Replace by ID
DELETE	/v1/app-configs/{id}	Delete — 204 on success

Application Assignments

Method	Path	Description
GET	/v1/app-assignments	List. Query: ?application_config_id=<id>&device_id=<id>&group_id=<id>
POST	/v1/app-assignments	Create — body: { application_config_id, device_id, group_id } — device_id or group_id required
GET	/v1/app-assignments/{id}	Get by ID
PUT	/v1/app-assignments/{id}	Replace by ID
DELETE	/v1/app-assignments/{id}	Delete — 204 on success

Reported Application Assignments *(device-originated — no POST/PUT)*

Method	Path	Description
GET	/v1/reported-app-assignments	List. Query: ?application_config_id=<id>&device_id=<id>

Method	Path	Description
GET	/v1/reported-app-assignments/{id}	Get by ID
DELETE	/v1/reported-app-assignments/{id}	Delete — 204 on success

OS Versions

Method	Path	Description
GET	/v1/os-versions	List all OS versions
POST	/v1/os-versions	Create — body: { <code>commit_hash*</code> , <code>orchestrator_version*</code> , <code>description</code> }
GET	/v1/os-versions/{id}	Get by ID
PUT	/v1/os-versions/{id}	Replace by ID
DELETE	/v1/os-versions/{id}	Delete — 204 on success

OS Assignments

Method	Path	Description
GET	/v1/os-assignments	List. Query: <code>?os_version_id=<id>&device_id=<id>&group_id=<id></code>
POST	/v1/os-assignments	Create — body: { <code>os_version_id</code> , <code>device_id</code> , <code>group_id</code> } — <code>device_id</code> or <code>group_id</code> required
GET	/v1/os-assignments/{id}	Get by ID
PUT	/v1/os-assignments/{id}	Replace by ID
DELETE	/v1/os-assignments/{id}	Delete — 204 on success

Reported OS Assignments (*device-originated* — no POST/PUT)

Method	Path	Description
GET	/v1/reported-os-assignments	List. Query: <code>?os_version_id=<id>&device_id=<id></code>
GET	/v1/reported-os-assignments/{id}	Get by ID

Method	Path	Description
DELETE	/v1/reported-os-assignments/{id}	Delete — 204 on success

Error responses

All errors return JSON: { "error": "<message>" }

Status	Meaning
404 Not Found	No resource with that ID
422 Unprocessable Entity	Validation failed (empty required field; missing <code>device_id/group_id</code>)
500 Internal Server Error	Database error